

Machine Learning Python Codes - Combined

Compiled from the 9 Python files provided by the user. Each section identifies what the code implements.

Contents: KNN, Simple Linear Regression, Perceptron, Naïve Bayes, K-Means, Density-Based Clustering, Gaussian Naïve Bayes email statistics, Logistic Regression, and Single Layer Perceptron.

1. K-Nearest Neighbours (KNN)

Source file: one.py

K-Nearest Neighbours (KNN): calculates Euclidean distances from test point (4,3), selects the 3 nearest neighbours, and predicts Red/Blue by majority vote.

```
import pandas as pd
import numpy as np
data=pd.read_csv("one.csv")
print(data)
distances=[]
test_point=(4,3)
X=data[["X","Y"]].values
z=data["Label"]
#print(X)
#print(test_point[0])

for i in range(len(X)):
    x=X[i][0]
    y=X[i][1]
    label=z[i]

    dist=((test_point[0]-x)**2 + (test_point[1]-y)**2)**0.5
    distances.append((dist, label))
distances.sort()
#print(distances)

nearest=distances[:3]
#print(nearest)
red_count=0
blue_count=0
for dist,label in nearest:
    if label=="Red":
        red_count+=1
    else:
        blue_count+=1
if red_count>blue_count:
    print("it belongs to red")
else:
    print("it belongs to blue")
```

2. Simple Linear Regression

Source file: two.py

Simple Linear Regression: manually calculates the slope and intercept using means, covariance-style numerator, and variance-style denominator.

```
import pandas as pd
import numpy as np
data=pd.read_csv("two.csv")
#print(data)
X=data["Marks in Class test"]
Y=data["Marks in Semester"]
b=0

n=len(X)
print(X)
print(Y)
q=sum(X)/n
s=sum(Y)/n

print(q)
print(s)
numerator=0
denominator=0
for i in range(n):
    numerator += (X[i]-q) * (Y[i]-s)
    denominator+=(X[i]-q)**2
t=numerator/denominator
b=s-(t*q)
print(b)
print("slope", t)
#print(numerator)
#print(denominator)
```

3. Perceptron Learning - Bipolar Output

Source file: three.py

Perceptron Learning (bipolar labels): repeatedly updates w_1 , w_2 and bias until all training examples are classified correctly.

```
import pandas as pd
import numpy as np
data=pd.read_csv("three.csv")
#print(data)
w1=0
w2=0
b=0
eta=1
print(data.columns)
X=data[["x1","x2"]].values
Y=data["y"].values

n=len(Y)
while True:
    error=0
    for i in range(n):
        x1=X[i][0]
        x2=X[i][1]
        z=(x1*w1+x2*w2)+b
        if z>0:
            prediction=1
        else:
            prediction=-1
        if prediction== Y[i]:
            print("Hurrah!")
        else:
            w1=w1+ eta*Y[i]*x1
            w2=w2+ eta*Y[i]*x2
            b=b+ eta*Y[i]
            error+=1

    if error ==0:
        break
print("final w1",w1)
print("final w2",w2)
print("final b",b)
```

4. Naive Bayes - Weather / Play Classification

Source file: four.py

Naive Bayes classification: computes prior and conditional probabilities for Play=Yes/No using Weather and Temperature, then compares class scores.

```
import pandas as pd
import numpy as np
data=pd.read_csv("four.csv")
print(data)

X=data["Play"]

n=len(X)
yes_count=0
no_count=0
for i in range(n):
    if X[i]=="Yes":
        yes_count+=1
    else:
        no_count+=1
total=len(X)
p_yes=yes_count/total
p_no=no_count/total
print(p_yes)
print(p_no)

y=data["Weather"]
sunny_count=0
cool_count=0
for i in range(n):
    if y[i]=="Sunny" and X[i]=="Yes":
        sunny_count+=1

#print(sunny_count)

z=data["Temperature"]
for i in range(n):
    if z[i]=="Cool" and X[i]=="Yes":
        cool_count+=1

print(cool_count)

p_sunny_yes=sunny_count/yes_count
p_cool_yes=cool_count/yes_count

print("P(Sunny|yes) =", p_sunny_yes)
print("P(Cool|Yes)" ,p_cool_yes)

sunny_no_count = 0
cool_no_count = 0

for i in range(n):
    if y[i] == "Sunny" and X[i] == "No":
        sunny_no_count += 1

    if z[i] == "Cool" and X[i] == "No":
        cool_no_count += 1

p_sunny_no = sunny_no_count / no_count
p_cool_no = cool_no_count / no_count

print("P(Sunny|No) =", p_sunny_no)
print("P(Cool|No) =", p_cool_no)

score_yes=p_yes * p_sunny_yes * p_cool_yes
score_no=p_no * p_sunny_no * p_cool_no

if score_yes > score_no:
    print("Yes game will happen")
else:
    print("Game will no happen")
```

5. K-Means Clustering

Source file: five.py

K-Means clustering: starts with the first two data points as centroids, assigns points by Euclidean distance, and recomputes centroids for two iterations.

```
import pandas as pd
import numpy as np
data=pd.read_csv("five.csv")
#print(data)
X=data[["X","Y"]].values
#print(X)

c1=X[0]
c2=X[1]
print(c1)
print(c2)
iteration=0

def distance(point,centroid):
    x1=point[0]
    y1=point[1]
    x2=centroid[0]
    y2=centroid[1]
    d=((x1-x2)**2 + (y1-y2)**2) **0.5
    return d

n=len(X)
for iteration in range(2):
    cluster1=[]
    cluster2=[]
    print("\nIteration" ,iteration + 1)

    for i in range(n):
        d1=distance(X[i],c1)
        d2=distance(X[i],c2)

        #print("Point=",X[i])
        #print("Distance from c1=" ,d1)
        #print("Distance from c2=" ,d2)

        if d1<d2 :
            cluster1.append(X[i])
        else:
            cluster2.append(X[i])
    print("cluster1:= ",cluster1)

    print("cluster2:= ",cluster2)
    x_sum=0
    y_sum=0
    for point in cluster1:
        x_sum=x_sum+point[0]
        y_sum=y_sum+point[1]
    c1=(x_sum/len(cluster1),y_sum/len(cluster1))

    x_sum=0
    y_sum=0
    for point in cluster2:
        x_sum=x_sum+point[0]
        y_sum=y_sum+point[1]
    c2=(x_sum/len(cluster2),y_sum/len(cluster2))
    print("new c1" ,c1)
    print("new c2" ,c2)
```

6. Density-Based Clustering (DBSCAN-style)

Source file: density.py

Density-based clustering (DBSCAN-style): finds neighbours within epsilon=2.5, identifies core points using minpts=3, and detects border points.

```
import pandas as pd
import numpy as np
data=pd.read_csv("density.csv")
#print(data)

epoch=2.5
minpts=3
X=data[["X", "Y"]].values
n=len(X)
def distance(point1,point2):
    x1=point1[0]
    x2=point2[0]

    y1=point1[1]
    y2=point2[1]
    return ((x1-x2)**2+(y1-y2)**2)**0.5
all_neighbors=[]
for i in range(n):
    neighbors=[]
    for j in range(n):
        d=distance(X[i],X[j])
        if i!=j and d<=epoch:
            neighbors.append(X[j])
    all_neighbors.append(neighbors)
    print(X[i])
    print(neighbors)

#print(all_neighbors)

core=[]
for i in range(n):
    if len(all_neighbors[i])>=minpts :
        core.append(X[i])
print("core",core)

for i in range(n):
    if len(all_neighbors[i])< minpts :
        for k in core:
            d=distance(k,X[i])
            if d<=epoch:
                print("Border Point: ",X[i])
                break
```

7. Gaussian Naive Bayes - Email Spam Statistics

Source file: email.py

Gaussian Naive Bayes preparation for email spam data: calculates class priors plus mean and variance of Word_Free, Word_Win, Length and Links for Spam and Not_Spam.

```
import pandas as pd
import numpy as np
import math
data = pd.read_csv("bayes.csv")
print(data)

X = data["Class"].values
n = len(X)

# ----- Prior Probability -----

spam_count = 0
not_spam_count = 0

for i in range(n):
    if X[i] == "Spam":
        spam_count += 1
    else:
        not_spam_count += 1

total = spam_count + not_spam_count

p_spam = spam_count / total
p_not_spam = not_spam_count / total

print("P(Spam) =", p_spam)
print("P(Not_Spam) =", p_not_spam)

# =====
#                      SPAM
# =====

# ----- Word_Free -----

free = data["Word_Free"].values

spam_free = []

for i in range(n):
    if X[i] == "Spam":
        spam_free.append(free[i])

print(spam_free)

mean_spam_free = sum(spam_free) / len(spam_free)
print("Spam Word_Free Mean =", mean_spam_free)

variance_spam_free = 0

for x in spam_free:
    variance_spam_free += (x - mean_spam_free) ** 2

variance_spam_free = variance_spam_free / len(spam_free)

print("Spam Word_Free Variance =", variance_spam_free)

# ----- Word_Win -----

win = data["Word_Win"].values

spam_win = []

for i in range(n):
    if X[i] == "Spam":
        spam_win.append(win[i])

print(spam_win)
```

```

mean_spam_win = sum(spam_win) / len(spam_win)
print("Spam Word_Win Mean =", mean_spam_win)

variance_spam_win = 0

for x in spam_win:
    variance_spam_win += (x - mean_spam_win) ** 2

variance_spam_win = variance_spam_win / len(spam_win)

print("Spam Word_Win Variance =", variance_spam_win)

# ----- Length -----

length = data["Length"].values

spam_length = []

for i in range(n):
    if X[i] == "Spam":
        spam_length.append(length[i])

print(spam_length)

mean_spam_length = sum(spam_length) / len(spam_length)
print("Spam Length Mean =", mean_spam_length)

variance_spam_length = 0

for x in spam_length:
    variance_spam_length += (x - mean_spam_length) ** 2

variance_spam_length = variance_spam_length / len(spam_length)

print("Spam Length Variance =", variance_spam_length)

# ----- Links -----

links = data["Links"].values

spam_links = []

for i in range(n):
    if X[i] == "Spam":
        spam_links.append(links[i])

print(spam_links)

mean_spam_links = sum(spam_links) / len(spam_links)
print("Spam Links Mean =", mean_spam_links)

variance_spam_links = 0

for x in spam_links:
    variance_spam_links += (x - mean_spam_links) ** 2

variance_spam_links = variance_spam_links / len(spam_links)

print("Spam Links Variance =", variance_spam_links)

# =====
#           NOT SPAM
# =====

# ----- Word_Free -----

not_spam_free = []

for i in range(n):
    if X[i] == "Not_Spam":
        not_spam_free.append(length[i])

print(not_spam_free)

mean_not_spam_free = sum(not_spam_free) / len(not_spam_free)
print("Not_Spam Word_Free Mean =", mean_not_spam_free)

```

```

variance_not_spam_free = 0

for x in not_spam_free:
    variance_not_spam_free += (x - mean_not_spam_free) ** 2

variance_not_spam_free = variance_not_spam_free / len(not_spam_free)

print("Not_Spam Word_Free Variance =", variance_not_spam_free)

# ----- Word_Win -----

not_spam_win = []

for i in range(n):
    if X[i] == "Not_Spam":
        not_spam_win.append(win[i])

print(not_spam_win)

mean_not_spam_win = sum(not_spam_win) / len(not_spam_win)
print("Not_Spam Word_Win Mean =", mean_not_spam_win)

variance_not_spam_win = 0

for x in not_spam_win:
    variance_not_spam_win += (x - mean_not_spam_win) ** 2

variance_not_spam_win = variance_not_spam_win / len(not_spam_win)

print("Not_Spam Word_Win Variance =", variance_not_spam_win)

# ----- Length -----

not_spam_length = []

for i in range(n):
    if X[i] == "Not_Spam":
        not_spam_length.append(length[i])

print(not_spam_length)

mean_not_spam_length = sum(not_spam_length) / len(not_spam_length)
print("Not_Spam Length Mean =", mean_not_spam_length)

variance_not_spam_length = 0

for x in not_spam_length:
    variance_not_spam_length += (x - mean_not_spam_length) ** 2

variance_not_spam_length = variance_not_spam_length / len(not_spam_length)

print("Not_Spam Length Variance =", variance_not_spam_length)

# ----- Links -----

not_spam_links = []

for i in range(n):
    if X[i] == "Not_Spam":
        not_spam_links.append(links[i])

print(not_spam_links)

mean_not_spam_links = sum(not_spam_links) / len(not_spam_links)
print("Not_Spam Links Mean =", mean_not_spam_links)

variance_not_spam_links = 0

for x in not_spam_links:
    variance_not_spam_links += (x - mean_not_spam_links) ** 2

variance_not_spam_links = variance_not_spam_links / len(not_spam_links)

print("Not_Spam Links Variance =", variance_not_spam_links)

```

8. Logistic Regression from Scratch

Source file: logistic_regression.py

Logistic Regression from scratch: implements sigmoid, hypothesis, binary cross-entropy cost, gradient descent, training, and prediction for 20 marks.

```
import pandas as pd
import numpy as np
import math
data=pd.read_csv("logistic.csv")
print(data)
X=data["Marks in Semester"].values/100
Y=data["y"].values
n=len(X)
def sigmoid(z):
    return 1/(1+math.exp(-z))

def hypothesis(x,w,b):
    z= w * x +b
    return sigmoid(z)

def cost_function(x,y,w,b):
    cost=0
    for i in range(n):
        y_hat=hypothesis(X[i],w,b)

        cost+=-(y[i]*math.log(y_hat)+(1-y[i])*math.log(1-y_hat))
    cost=cost/n
    return cost

def gradient_descent(x,y,w,b, learning_rate, epochs):
    for epoch in range(epochs):
        dw=0
        db=0
        for i in range(n):
            y_hat=hypothesis(x[i],w,b)
            error=y_hat-y[i]
            dw+=error*x[i]
            db+=error
        dw=dw/n
        db=db/n
        w=w- learning_rate*dw
        b=b- learning_rate*db
    return w,b

def linear_regression(x,y):
    w=0
    b=0
    learning_rate=0.01
    epochs=1000
    w,b=gradient_descent(x,y,w,b, learning_rate, epochs)
    return w,b
w,b=linear_regression(X,Y)
print("w=",w)
print("b=",b)

marks=20/100
probability=hypothesis(marks,w,b)
if probability>=0.5:
    print("pass")
else:
    print("fail")
```

9. Single Layer Perceptron - AND Gate

Source file: slp.py

Single Layer Perceptron for an AND-gate-style dataset: predicts 0/1 and updates weights and bias until there are no classification errors.

```
import pandas as pd
import numpy as np
data=pd.read_csv("and.csv")
print(data)

w1=0
w2=0
b=0
eta=1
print(data.columns)
X=data[["x1", "x2"]].values
Y=data["Desired output"].values

n=len(Y)
while True:
    error=0
    for i in range(n):
        x1=X[i][0]
        x2=X[i][1]
        z=(x1*w1+x2*w2)+b
        #Model predicting
        if z>=0:
            y=1
        else:
            y=0
        if y == Y[i]:
            print("Hurrah!")
        else:
            w1=w1+ eta*(Y[i]-y)*x1
            w2=w2+ eta*(Y[i]-y)*x2
            b=b+ eta*(Y[i]-y)
            error+=1

    if error==0:
        break
print("final w1",w1)
print("final w2",w2)
print("final b",b)
```